

CRÉER UN SITE WEB

Toutes les commandes étape par étape

Linux / Mac • Windows PowerShell • Git • NPM • PHP • Python

Édition June 2026

■ HTML/CSS

■ Node.js / NPM

■ Git & GitHub

■ Apache/Nginx

■ SSL / HTTPS

TABLE DES MATIÈRES

Étape 1 — Préparer l'environnement de travail

- Installer un éditeur de code (VS Code)
- Installer Git
- Installer Node.js & NPM

Étape 2 — Créer la structure du projet

- Initialiser le dossier
- Créer les fichiers de base (HTML, CSS, JS)
- Structure type d'un projet

Étape 3 — Initialiser Git et versionner le code

- Init, add, commit
- Créer une branche
- Lier à GitHub/GitLab

Étape 4 — Développer avec un framework CSS

- Bootstrap via NPM ou CDN
- Tailwind CSS
- Compilation

Étape 5 — Créer et gérer un site WordPress (CMS)

- Installer XAMPP / LAMP
- Créer la base de données
- Configurer wp-config.php

Étape 6 — Créer un site avec Python (Django/Flask)

- Environnement virtuel
- Installer Django ou Flask
- Lancer le serveur de dev

Étape 7 — Créer un site avec Node.js / Express

- Init NPM
- Installer Express
- Lancer l'application

Étape 8 — Tester et valider le site

- Tester en local
- Outils de validation
- Tests de performance

Étape 9 — Mettre en ligne (Déploiement)

- FTP avec lftp
- Déploiement sur Netlify / Vercel
- Déploiement sur un VPS Linux

Étape 10 — Sécuriser et maintenir le site

- Configurer HTTPS / SSL
- Permissions fichiers

- Mises à jour & sauvegardes

ÉTAPE 1

PRÉPARER L'ENVIRONNEMENT

Installer tous les outils nécessaires avant d'écrire la première ligne de code

1.1 — Installer VS Code (éditeur de code)

Télécharger depuis <https://code.visualstudio.com>. Commandes utiles après installation :

■ Linux (Debian/Ubuntu)

```
# Télécharger et installer VS Code
$ sudo apt update
$ sudo apt install wget gpg
$ wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg
--dearmor > packages.microsoft.gpg
$ sudo install -o root -g root -m 644 packages.microsoft.gpg
/etc/apt/trusted.gpg.d/
$ sudo sh -c 'echo "deb [arch=amd64]
https://packages.microsoft.com/repos/vscode stable main" >
/etc/apt/sources.list.d/vscode.list'
$ sudo apt update
$ sudo apt install code
# Lancer VS Code
$ code .
# (le point "." ouvre le dossier courant)
```

■ Extensions VS Code recommandées

```
# Installer des extensions depuis le terminal
$ code --install-extension esbenp.prettier-vscode
$ code --install-extension dbaeumer.vscode-eslint
$ code --install-extension ritwickdey.liveserver
$ code --install-extension formulahendry.auto-rename-tag
$ code --install-extension ecmel.vscode-html-css
```

1.2 — Installer Git

```
# Linux (Debian/Ubuntu)
$ sudo apt install git
# macOS (avec Homebrew)
$ brew install git
# Windows – télécharger sur https://git-scm.com
# Vérifier l'installation
$ git --version
git version 2.43.0
# Configuration initiale obligatoire
$ git config --global user.name "Votre Nom"
$ git config --global user.email "votre@email.com"
$ git config --global core.editor "code --wait"
# Vérifier la config
$ git config --list
```

1.3 — Installer Node.js et NPM

```
# Linux – via NodeSource (recommandé)
$ curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
$ sudo apt install -y nodejs
# macOS
$ brew install node
# Vérifier les versions
$ node --version
v20.11.0
$ npm --version
10.2.4
# Mettre à jour NPM
$ npm install -g npm@latest
```

■ Conseil

Utilisez **nvm** (Node Version Manager) pour gérer plusieurs versions de Node.js : `curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash`

ÉTAPE 2

CRÉER LA STRUCTURE DU PROJET

Organiser ses fichiers correctement dès le départ

2.1 — Initialiser le dossier de projet

```
# Créer le dossier du projet et y entrer
$ mkdir mon-site-web
$ cd mon-site-web
# Créer les sous-dossiers principaux
$ mkdir css js images assets
# Créer les fichiers de base
$ touch index.html
$ touch css/style.css
$ touch js/main.js
# Ouvrir le projet dans VS Code
$ code .
```

2.2 — Structure type d'un projet web

```
# Afficher l'arborescence du projet
$ tree mon-site-web
mon-site-web/
├── index.html (page principale)
├── about.html (page À propos)
├── contact.html (page Contact)
├── css/
│   ├── style.css (styles globaux)
│   └── responsive.css (media queries)
├── js/
│   └── main.js (scripts)
├── images/ (photos, icônes)
└── assets/ (polices, PDF...)
# Installer tree si absent
$ sudo apt install tree # Linux
$ brew install tree # macOS
```

2.3 — Créer un fichier HTML de base

```
# Contenu minimal de index.html (coller dans VS Code)
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Description de ma page">
<title>Mon Site Web</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<header></header>
<main></main>
<footer></footer>
<script src="js/main.js"></script>
</body>
</html>
```

■ Live Server

Dans VS Code, clic droit sur index.html > 'Open with Live Server' pour voir les modifications en temps réel dans le navigateur.

ÉTAPE 3

GIT — VERSIONNER LE CODE

Sauvegarder chaque avancée et travailler en équipe

3.1 — Initialiser Git dans le projet

```
# Initialiser le dépôt Git
$ git init
Initialized empty Git repository in .git/
# Créer le fichier .gitignore (fichiers à ne PAS versionner)
$ touch .gitignore
# Contenu type du .gitignore :
node_modules/
.env
.DS_Store
dist/
*.log
```

3.2 — Les commandes Git essentielles

```
# Vérifier l'état du projet
$ git status
# Ajouter tous les fichiers modifiés
$ git add .
# Ou ajouter un fichier spécifique
$ git add index.html
# Enregistrer les modifications (commit)
$ git commit -m "Initial commit : structure de base"
# Voir l'historique des commits
$ git log --oneline
a1b2c3d Initial commit : structure de base
# Créer une nouvelle branche (ex: pour une fonctionnalité)
$ git checkout -b feature/menu-navigation
# Fusionner la branche dans main
$ git checkout main
$ git merge feature/menu-navigation
```

3.3 — Lier au dépôt distant (GitHub / GitLab)


```
# 1. Créer un compte sur https://github.com
# 2. Créer un nouveau dépôt (repo) vide sur GitHub
# 3. Lier le projet local au dépôt distant :
$ git remote add origin https://github.com/votre-user/mon-site-web.git
# Envoyer le code sur GitHub
$ git push -u origin main
# Commandes push/pull quotidiennes
$ git push # envoyer ses commits
$ git pull # récupérer les mises à jour
$ git clone https://github.com/user/repo.git # cloner un projet
```

■ Ne jamais committer

Ne mettez jamais de mots de passe, clés API ou informations sensibles dans Git. Utilisez un fichier .env et ajoutez-le au .gitignore.

ÉTAPE 4

FRAMEWORK CSS — BOOTSTRAP & TAILWIND

Gagner du temps avec des bibliothèques de styles professionnelles

4.1 — Bootstrap via NPM

```
# Initialiser NPM dans le projet
$ npm init -y
# Installer Bootstrap
$ npm install bootstrap
# Chemin vers les fichiers installés :
node_modules/bootstrap/dist/css/bootstrap.min.css
node_modules/bootstrap/dist/js/bootstrap.bundle.min.js
# Lier dans index.html (dans <head>) :
<link rel="stylesheet"
href="node_modules/bootstrap/dist/css/bootstrap.min.css">
# Lier le JS Bootstrap (avant </body>) :
<script
src="node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
```

CDN Bootstrap (sans NPM) : coller dans <head> :

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
```

4.2 — Tailwind CSS

```
# Installer Tailwind CSS
$ npm install -D tailwindcss
$ npx tailwindcss init
# Configurer tailwind.config.js (sources HTML/JS) :
content: ["/src/**/*.html,js"],
# Créer src/input.css avec :
@tailwind base;
@tailwind components;
@tailwind utilities;
# Compiler le CSS (mode dev avec watch)
$ npx tailwindcss -i ./src/input.css -o ./dist/output.css --watch
# Compiler pour la production (minifié)
$ npx tailwindcss -i ./src/input.css -o ./dist/output.css --minify
```

■ Conseil pratique

Pour les débutants, Bootstrap est plus simple à prendre en main. Tailwind est plus flexible mais demande de comprendre les classes utilitaires.

ÉTAPE 5

SITE WORDPRESS (CMS)

Installer et configurer WordPress en local puis en production

5.1 — Installer XAMPP / LAMP (serveur local)

```
# Linux – Installer Apache, MySQL, PHP (LAMP stack)
$ sudo apt update
$ sudo apt install apache2 mysql-server php php-mysql php-zip php-curl
php-gd
# Démarrer les services
$ sudo systemctl start apache2
$ sudo systemctl start mysql
# Activer au démarrage
$ sudo systemctl enable apache2
$ sudo systemctl enable mysql
# Vérifier Apache : ouvrir http://localhost dans le navigateur
# Le dossier web est : /var/www/html/
```

5.2 — Créer la base de données MySQL

```
# Se connecter à MySQL
$ sudo mysql -u root -p
# Dans le shell MySQL :
mysql> CREATE DATABASE wordpress_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
mysql> CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'mot_de_passe';
mysql> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';
mysql> FLUSH PRIVILEGES;
mysql> EXIT;
```

5.3 — Installer WordPress

```
# Télécharger WordPress
$ cd /var/www/html
$ sudo wget https://fr.wordpress.org/latest-fr_FR.tar.gz
$ sudo tar -xvzf latest-fr_FR.tar.gz
$ sudo mv wordpress mon-site
# Donner les permissions à Apache
$ sudo chown -R www-data:www-data /var/www/html/mon-site
$ sudo chmod -R 755 /var/www/html/mon-site
# Configurer wp-config.php
$ cd mon-site
$ sudo cp wp-config-sample.php wp-config.php
$ sudo nano wp-config.php
# Modifier ces 3 lignes dans wp-config.php :
define('DB_NAME', 'wordpress_db');
define('DB_USER', 'wp_user');
define('DB_PASSWORD', 'mot_de_passe');
# Finaliser via navigateur : http://localhost/mon-site
```

■ WP-CLI

Gérer WordPress en ligne de commande : `wp plugin install contact-form-7 --activate` | `wp theme install astra --activate` | `wp core update`

ÉTAPE 6

SITE PYTHON — DJANGO / FLASK

Créer un site dynamique avec Python

6.1 — Créer un environnement virtuel Python

```
# Créer le dossier projet
$ mkdir mon-site-python && cd mon-site-python
# Créer l'environnement virtuel
$ python3 -m venv venv
# Activer l'environnement
$ source venv/bin/activate # Linux/macOS
$ venv\Scripts\activate # Windows
# Vérifier que l'env est actif (préfixe (venv) dans le terminal)
(venv) $
# Désactiver l'environnement
$ deactivate
```

6.2 — Site avec Flask (micro-framework)

```
# Installer Flask
(venv) $ pip install flask
# Créer app.py :
from flask import Flask, render_template
app = Flask(__name__)
@app.route("/")
def index():
    return render_template("index.html")
if __name__ == "__main__":
    app.run(debug=True)
# Lancer le serveur Flask
(venv) $ python app.py
* Running on http://127.0.0.1:5000
```

6.3 — Site avec Django (framework complet)

```
# Installer Django
(venv) $ pip install django
# Créer un projet Django
(venv) $ django-admin startproject monsite .
# Créer une application
(venv) $ python manage.py startapp pages
# Appliquer les migrations (base de données)
(venv) $ python manage.py migrate
# Créer un super-utilisateur (admin)
(venv) $ python manage.py createsuperuser
# Lancer le serveur de développement
(venv) $ python manage.py runserver
Starting development server at http://127.0.0.1:8000/
# Sauvegarder les dépendances
(venv) $ pip freeze > requirements.txt
# Réinstaller sur un autre poste :
(venv) $ pip install -r requirements.txt
```

ÉTAPE 7

SITE NODE.JS / EXPRESS

Créer un serveur web avec JavaScript côté serveur

7.1 — Initialiser le projet Node.js

```
# Créer le dossier et initialiser NPM
$ mkdir mon-site-node && cd mon-site-node
$ npm init -y
# Installer Express.js
$ npm install express
# Installer nodemon (rechargement automatique en dev)
$ npm install -D nodemon
# Ajouter dans package.json, section "scripts" :
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

7.2 — Créer le serveur Express

```
# Contenu de index.js :
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;
// Servir les fichiers statiques
app.use(express.static('public'));
app.use(express.json());
// Route principale
app.get('/', (req, res) => {
  res.sendFile(__dirname + '/public/index.html');
});
app.listen(PORT, () => {
  console.log(`Serveur sur http://localhost:${PORT}`);
});
# Lancer en mode développement
$ npm run dev
Serveur sur http://localhost:3000
```


■ Variables d'environnement

Installez dotenv pour gérer les variables sensibles : `npm install dotenv` — puis créez `.env` avec vos variables (`PORT=3000`, `DB_URL=...`) et ajoutez `.env` dans `.gitignore`.

ÉTAPE 8

TESTER ET VALIDER LE SITE

S'assurer que tout fonctionne avant la mise en ligne

8.1 — Tester en local

```
# Tester avec un simple serveur HTTP Python
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/)
# Tester avec Live Server (VS Code)
# Clic droit sur index.html > Open with Live Server
# Vérifier les liens cassés (avec wget)
$ wget --spider -r -nd --delete-after http://localhost:8080 2>&1 | grep
-E "broken|ERROR"
```

8.2 — Valider le code HTML et CSS

```
# Installer le validateur HTML W3C en ligne de commande
$ npm install -g html-validate
# Valider un fichier HTML
$ html-validate index.html
# Valider le CSS avec stylelint
$ npm install -g stylelint stylelint-config-standard
$ stylelint "css/**/*.css"
# Formater automatiquement le code avec Prettier
$ npm install -g prettier
$ prettier --write "**/*.html" "**/*.css" "**/*.js"
```

8.3 — Tester les performances

```
# Lighthouse via Chrome CLI
$ npm install -g lighthouse
$ lighthouse http://localhost:8080 --output html --output-path
./rapport.html
# Tester la vitesse de chargement avec curl
$ curl -o /dev/null -s -w "Temps total: %{time_total}s\n"
http://localhost:8080
# Vérifier la compatibilité mobile (DevTools Chrome)
# F12 > Icône mobile > choisir un appareil
```

■ Checklist rapide avant déploiement

- Tous les liens fonctionnent | ■ Le site est responsive | ■ Pas d'erreurs dans la console navigateur |
- Images optimisées | ■ HTML/CSS validés | ■ Performance Lighthouse > 80

ÉTAPE 9

DÉPLOIEMENT — METTRE EN LIGNE

Publier le site sur internet pour le monde entier

9.1 — Déploiement FTP sur un hébergement mutualisé

```
# Installer lftp (client FTP en ligne de commande)
$ sudo apt install lftp
# Se connecter et uploader les fichiers
$ lftp -u utilisateur,motdepasse ftp.monhebergeur.com
# Dans le shell lftp :
lftp> cd public_html
lftp> mirror -R /chemin/local/mon-site-web/ .
# -R = reverse (upload du local vers le serveur)
# Uploader un seul fichier :
lftp> put index.html
lftp> bye
```

9.2 — Déploiement sur Netlify (sites statiques)

```
# Installer la CLI Netlify
$ npm install -g netlify-cli
# Se connecter
$ netlify login
# Déployer en mode prévisualisation
$ netlify deploy --dir=.
# Déployer en production
$ netlify deploy --dir=. --prod
Website URL: https://mon-site.netlify.app
# Déploiement automatique depuis GitHub :
# Se fait via l'interface Netlify : lier un repo GitHub
# Chaque git push déclenche un déploiement automatique
```

9.3 — Déploiement sur un VPS Linux (production)

```

# Se connecter au VPS via SSH
$ ssh utilisateur@ip-du-serveur
# Installer Nginx (serveur web)
$ sudo apt update && sudo apt install nginx
$ sudo systemctl start nginx
# Copier les fichiers du site
$ sudo mkdir -p /var/www/mon-site
$ sudo scp -r /chemin/local/* utilisateur@ip:/var/www/mon-site/
# Configurer Nginx pour servir le site
$ sudo nano /etc/nginx/sites-available/mon-site
# Contenu de la config Nginx :
server {
    listen 80;
    server_name mon-domaine.com www.mon-domaine.com;
    root /var/www/mon-site;
    index index.html;
}
# Activer la configuration
$ sudo ln -s /etc/nginx/sites-available/mon-site
/etc/nginx/sites-enabled/
$ sudo nginx -t # Vérifier la config
$ sudo systemctl reload nginx

```

9.4 — Déployer une app Node.js avec PM2

```

# Installer PM2 (gestionnaire de processus)
$ npm install -g pm2
# Démarrer l'application
$ pm2 start index.js --name "mon-site"
# Démarrage automatique au reboot
$ pm2 startup
$ pm2 save
# Commandes PM2 utiles
$ pm2 list # lister les apps
$ pm2 logs mon-site # voir les logs
$ pm2 restart mon-site # redémarrer
$ pm2 stop mon-site # arrêter

```

ÉTAPE 10

SÉCURITÉ ET MAINTENANCE

Protéger le site et le maintenir en bonne santé

10.1 — Configurer HTTPS avec Let's Encrypt

```
# Installer Certbot (pour Nginx)
$ sudo apt install certbot python3-certbot-nginx
# Obtenir et installer le certificat SSL
$ sudo certbot --nginx -d mon-domaine.com -d www.mon-domaine.com
# Tester le renouvellement automatique
$ sudo certbot renew --dry-run
# Vérifier la date d'expiration du certificat
$ sudo certbot certificates
Expiry Date: 2025-09-01 (VALID: 89 days)
```

10.2 — Permissions des fichiers

```
# Règle d'or des permissions web
$ sudo find /var/www/mon-site -type d -exec chmod 755 {} \;
$ sudo find /var/www/mon-site -type f -exec chmod 644 {} \;
# Propriétaire = www-data (Apache/Nginx)
$ sudo chown -R www-data:www-data /var/www/mon-site
# Vérifier les permissions
$ ls -la /var/www/mon-site
drwxr-xr-x www-data www-data (répertoire: 755)
-rw-r--r-- www-data www-data (fichier: 644)
```

10.3 — Sauvegardes automatiques

```

# Script de sauvegarde des fichiers
$ nano backup.sh
#!/bin/bash
DATE=$(date +%Y-%m-%d)
tar -czf /backup/site_$DATE.tar.gz /var/www/mon-site
echo "Sauvegarde effectuée : site_$DATE.tar.gz"

$ chmod +x backup.sh
# Automatiser avec cron (chaque jour à 2h du matin)
$ crontab -e
0 2 * * * /home/user/backup.sh
# Sauvegarde de base de données MySQL
$ mysqldump -u root -p wordpress_db > backup_db_$(date +%Y%m%d).sql

```

10.4 — Mises à jour et surveillance

```

# Mettre à jour le système
$ sudo apt update && sudo apt upgrade -y
# Surveiller les logs du serveur Nginx
$ sudo tail -f /var/log/nginx/access.log
$ sudo tail -f /var/log/nginx/error.log
# Vérifier l'espace disque
$ df -h
# Vérifier la mémoire utilisée
$ free -h
# Mettre à jour les packages NPM d'un projet
$ npm outdated # voir les packages obsolètes
$ npm update # mettre à jour
# Mettre à jour WordPress en ligne de commande (WP-CLI)
$ wp core update
$ wp plugin update --all
$ wp theme update --all

```

■ Sauvegardez AVANT de mettre à jour

Faites toujours une sauvegarde complète (fichiers + base de données) avant d'effectuer des mises à jour en production.

RÉCAPITULATIF — AIDE-MÉMOIRE

■ Commandes Git les plus utilisées

```
$ git init | git clone URL | git status | git add . | git commit -m "msg"
$ git push | git pull | git checkout -b branche | git merge branche
$ git log --oneline | git diff | git stash | git stash pop
```

■ Commandes NPM essentielles

```
$ npm init -y # initialiser un projet
$ npm install package # installer une dépendance
$ npm install -D package # dépendance de développement
$ npm run dev # lancer le script "dev"
$ npm run build # compiler pour la production
$ npm update # mettre à jour les packages
```

■ Commandes Linux utiles pour le web

```
$ mkdir dossier # créer un dossier
$ touch fichier.html # créer un fichier vide
$ cp source destination # copier un fichier
$ mv source destination # déplacer/renommer
$ rm -rf dossier # supprimer (ATTENTION : irréversible)
$ nano fichier.txt # éditer un fichier (Ctrl+X pour quitter)
$ cat fichier.txt # afficher le contenu
$ grep "texte" fichier.txt # rechercher dans un fichier
$ chmod 755 script.sh # changer les permissions
$ curl https://url.com # tester une URL
$ wget https://url.com # télécharger un fichier
$ ssh user@ip-serveur # connexion SSH au serveur
```

■ Services système (systemctl)

```
$ sudo systemctl start nginx # démarrer Nginx
$ sudo systemctl stop nginx # arrêter Nginx
$ sudo systemctl restart nginx # redémarrer Nginx
$ sudo systemctl status nginx # voir le statut
$ sudo systemctl enable nginx # activer au démarrage
```


■ Ce guide vous accompagne de l'installation de l'environnement jusqu'à la mise en ligne. Pratiquez chaque commande, expérimentez, et n'ayez pas peur des erreurs : elles font partie de l'apprentissage !